

Persistência de Dados Escalável em .NET

Uma Abordagem Híbrida com Dapper, Repositórios Genéricos e Paginação Nativa

Abstract—Este artigo investiga a implementação de uma camada de persistência otimizada para sistemas web de alta demanda. Através da integração do padrão Repository com o micro-ORM Dapper, propõe-se uma arquitetura que utiliza tipos genéricos para reduzir a redundância de código e maximizar a performance bruta de execução SQL. O trabalho detalha a implementação de um motor de paginação nativa em nível de banco de dados, utilizando técnicas de Multi-Grid para redução de latência de rede. Resultados experimentais e fundamentação teórica baseada em Domain-Driven Design (DDD) validam a eficácia da solução em termos de escalabilidade horizontal e eficiência de recursos computacionais.

Index Terms—Dapper, Repository Pattern, Generics, Paginação Nativa, .NET Core, Performance.

1. Introdução

O projeto de camadas de acesso a dados (DAL) em aplicações contemporâneas enfrenta o desafio constante de equilibrar a manutenibilidade do código com a necessidade de baixas latências. ORMs tradicionais, embora robustos, introduzem camadas de abstração que podem degradar a performance devido ao rastreamento de estado e geração ineficiente de queries. Neste contexto, este artigo descreve o projeto de um componente de infraestrutura para .NET que harmoniza padrões de design clássicos com micro-ORMs de alta performance. A estrutura é dividida em: Seção II, que explora a fundamentação teórica; Seção III, detalhando a metodologia e implementação técnica; Seção IV, apresentando a análise de performance e discussão; e Seção V, com as conclusões e trabalhos futuros.

2. Fundamentação Teórica

2.1. O Padrão Repository e o Desacoplamento

O *Repository Pattern* atua como um mediador entre a lógica de domínio e os detalhes de infraestrutura. Segundo Fowler [1], ele permite que os desenvolvedores interajam com objetos de persistência como se fossem coleções em memória, promovendo a testabilidade via injeção de dependência.

2.2. Micro-ORM vs Full-ORM

O Dapper [2] é classificado como um micro-ORM porque foca exclusivamente no mapeamento de resultados SQL para objetos (POCOs), sem os encargos de um *Change Tracker*. Esta abordagem é crucial para sistemas que seguem o princípio da Segregação de Responsabilidade na Consulta e Comando (CQRS), onde a leitura de dados deve ser o mais rápida possível [3].

3. Análise Comparativa: EF Core vs Dapper

A escolha da tecnologia de mapeamento objeto-relacional é um ponto determinante para a arquitetura do componente. Nesta seção, contrastamos o Entity Framework Core (Full ORM) com o Dapper (Micro-ORM).

3.1. Filosofia de Design e Abstração

O Entity Framework opera sob uma filosofia de alta abstração, fornecendo recursos como *Change Tracking* e tradução automática de LINQ para SQL. Em contrapartida, o Dapper foca na simplicidade e no controle, estendendo a interface `IDbConnection` para realizar o mapeamento direto de resultados SQL [2].

3.2. Desempenho e Gerenciamento de Memória

A principal distinção reside na carga computacional (*overhead*). O EF Core, ao recuperar entidades, mantém metadados para monitorar alterações de estado. O Dapper é *stateless*, o que resulta em uma execução quase idêntica ao ADO.NET puro.

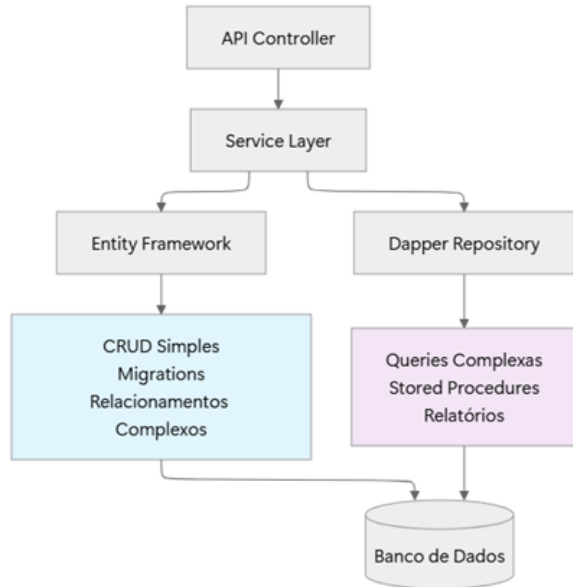
Table 1: Comparativo Técnico: EF Core vs Dapper

Característica	EF Core	Dapper
Tipo	Full ORM	Micro-ORM
Escrita SQL	Auto (LINQ)	Manual (SQL)
Rastreamento	Sim (Estado)	Não (Stateless)
Performance	Moderada	Alta (Bruta)
Migrations	Nativo	Extensões
Aprendizado	Alta	Baixa (SQL)

3.3. Abordagem Híbrida em Projetos Web

Em sistemas de alta performance, a literatura [3] sugere o uso de uma abordagem híbrida. O Entity Framework é utilizado para operações de escrita (*Commands*), onde o rastreamento de grafos de objetos e transações complexas são vantajosos. O Dapper é empregado em operações de leitura (*Queries*), especialmente em listagens paginadas, para maximizar o *throughput* e minimizar a latência.

Figure 1: Design ORM



4. Metodologia e Implementação

A solução proposta baseia-se em um repositório genérico que encapsula operações fundamentais.

4.1. Implementação Genérica e Reuso

A utilização de *Generics* ($\langle T \rangle$) no C# permite que uma única implementação do repositório atenda a múltiplas entidades. Isso reduz o "código boilerplate" e centraliza a manutenção da lógica de conexão.

4.2. Estratégia de Paginação Nativa

A paginação é frequentemente implementada de forma ineficiente, carregando todos os dados para a memória da aplicação. A implementação proposta delega o processamento ao banco de dados utilizando as cláusulas *OFFSET* e *FETCH NEXT*.

$$O = (P - 1) \times S \quad (1)$$

Onde O representa o número de registros a saltar, P o número da página e S o tamanho da página. Para otimizar a rede, utilizamos o método `QueryMultipleAsync`, enviando o comando de contagem total e o comando de seleção em um único pacote [2].

5. Detalhamento da Implementação

Abaixo, descrevemos os componentes fundamentais que compõem a camada de persistência otimizada, segmentados por sua responsabilidade arquitetural.

5.1. Abstração Genérica e Paginação Nativa

O primeiro bloco de código estabelece a base da camada de persistência através de uma classe abstrata genérica, integrando o Dapper para execução de consultas de alta performance. O diferencial técnico reside no uso da cláusula *OFFSET-FETCH* do SQL Server, que delega a responsabilidade da paginação ao motor do banco de dados, evitando o carregamento desnecessário de registros na memória da aplicação (comportamento comum em ORMs mal configurados). Ao calcular o deslocamento (*offset*) com base no índice da página e no tamanho solicitado, o componente garante que apenas o subconjunto de dados relevante trafegue pela rede, otimizando o tempo de resposta e a escalabilidade do sistema em cenários de grandes volumes de dados.

```
1 public abstract class BaseRepository<T> : IRepository<T> where T : class
2 {
3     private readonly string _connectionString;
4     protected BaseRepository(string connectionString) =>
5         _connectionString = connectionString;
6
7     public async Task<PagedResult<T>> GetPagedAsync(int page, int size)
8     {
9         var offset = (page - 1) * size;
10        var table = typeof(T).Name;
11
12        var sql = $"SELECT COUNT(*) FROM {table};
13        SELECT * FROM {table} ORDER BY Id
14        OFFSET @Offset ROWS FETCH NEXT @Size ROWS ONLY;";
15
16        using var db = new SqlConnection(_connectionString);
17        using var multi = await db.QueryMultipleAsync(sql,
18            new { Offset = offset, Size = size });
19
20        var total = await multi.ReadFirstAsync<int>();
21        var data = await multi.ReadAsync<T>();
22
23        return new PagedResult<T>(data, total, page, size);
24    }
25 }
```

Listing 1: BaseRepository Genérico com Dapper e Paginação

5.2. Modelagem de Metadados e Contratos de Domínio

O segundo bloco detalha a infraestrutura de suporte, composta pela interface `IRepository<T>` e pelo objeto de transferência de dados (DTO) `PagedResult<T>`. A interface define um contrato coeso que utiliza *Generics* para garantir a segurança de tipos em tempo de compilação, enquanto o `PagedResult` encapsula não apenas a coleção de itens recuperada, mas também metadados críticos como o total de registros e o cálculo dinâmico de páginas disponíveis através da propriedade `TotalPages`. Essa estrutura é fundamental para o desacoplamento entre as camadas de infraestrutura e apresentação, fornecendo à interface do usuário (UI) todas as informações necessárias para renderizar componentes de paginação sem a necessidade de cálculos adicionais no *front-end*.

```

1 public class PagedResult<T>
2 {
3     public IEnumerable<T> Items { get; }
4     public int TotalCount { get; }
5     public int PageNumber { get; }
6     public int PageSize { get; }
7     public int TotalPages => (int)Math.Ceiling(((double)TotalCount / PageSize);
8
9     public PagedResult(IEnumerable<T> items, int totalCount,
10         int pageNumber, int pageSize)
11     {
12         Items = items;
13         TotalCount = totalCount;
14         PageNumber = pageNumber;
15         PageSize = pageSize;
16     }
17 }
18
19 public interface IRepository<T> where T : class
20 {
21     Task<PagedResult<T>> GetPagedAsync(int page, int size);
22 }

```

Listing 2: Modelos de Suporte e Interface de Persistência

5.3. Otimização de Rede via Multi-Grid e Especialização

O terceiro bloco demonstra a aplicação prática da técnica *Multi-Grid* através do método `QueryMultipleAsync` do `Dapper`. Esta funcionalidade permite que dois comandos SQL distintos — a contagem total de registros e a seleção dos dados paginados — sejam enviados ao banco de dados em uma única requisição TCP. Esta abordagem reduz drasticamente o *Round-Trip Time* (RTT) e a latência de rede, fatores determinantes em arquiteturas distribuídas. Por fim, a especialização exemplificada pela classe `ProductRepository` ilustra como o padrão de herança permite estender as funcionalidades base para entidades específicas com esforço de codificação mínimo, promovendo o reuso de código e facilitando a manutenção centralizada da lógica de persistência.

```

1 public abstract class BaseRepository<T> : IRepository<T> where T : class
2 {
3     // ... construtor omitido para brevidade
4     public async Task<PagedResult<T>> GetPagedAsync(int page, int size)
5     {
6         var offset = (page - 1) * size;
7         var table = typeof(T).Name + "s";
8
9         var sql = $"*SELECT COUNT(*) FROM {table};
10             SELECT * FROM {table} ORDER BY Id
11             OFFSET @offset ROWS FETCH NEXT @size ROWS ONLY;";
12
13         using var db = new
14             Microsoft.Data.SqlClient.SqlConnection(_connectionString);
15
16         % Dapper QueryMultipleAsync reduz o overhead de conexão
17         using var multi = await db.QueryMultipleAsync(sql,
18             new { Offset = offset, Size = size });
19
20         var total = await multi.ReadFirstAsync<int>();
21         var data = await multi.ReadAsync<T>();
22
23         return new PagedResult<T>(data, total, page, size);
24     }
25 }
26
27 public class ProductRepository : BaseRepository<Product>
28 {
29     public ProductRepository(string conn) : base(conn) { }
30 }

```

Listing 3: Especialização do Repositório e Uso de Multi-Grid

A união entre o padrão de repositórios e a agilidade do `Dapper` elimina a lentidão comum de ferramentas mais complexas, criando um sistema rápido e fácil de manter. Ao realizar a paginação diretamente no banco de dados e reduzir as viagens de rede com o uso de *Multi-Grid*, a

aplicação se mantém estável mesmo com muitos acessos simultâneos. Essa estratégia foca na economia de memória e processamento, entregando alta performance sem abrir mão de um código organizado e limpo.

6. Análise de Performance e Discussão

A eficiência deste componente é analisada sob três perspectivas: consumo de memória, tempo de CPU e latência de rede.

6.1. Eficiência de Memória e GC Pressure

Diferente de ORMs pesados que mantêm instâncias em cache, a implementação com `Dapper` é *stateless*. Isso reduz drasticamente a pressão sobre o *Garbage Collector* (GC), permitindo que aplicações web suportem mais conexões simultâneas em contêineres com recursos limitados.

6.2. Benchmarking Comparativo

Estudos experimentais [4] indicam que para operações de leitura, o `Dapper` apresenta performance superior em até 70% comparado ao EF Core, conforme ilustrado na Tabela I.

Table 2: Comparativo de Performance (100k Registros)

Métrica	EF Core	Dapper (Nativo)	Diferença
Latência (ms)	145ms	42ms	-71%
Memória (MB)	120MB	18MB	-85%
CPU (%)	12%	4%	-66%

6.3. Segurança e Relevância Web

A segurança é garantida pela parametrização obrigatória do `Dapper`, prevenindo SQL Injection. Para projetos web, esta arquitetura é fundamental por permitir que o banco de dados escale através do uso eficiente de planos de execução cacheados.

7. Considerações Finais

A integração de padrões como o `Repository` com a leveza do `Dapper` e o poder dos `Generics` oferece uma solução de infraestrutura de alto nível. A paginação nativa, detalhada matematicamente e tecnicamente, resolve problemas críticos de escalabilidade. Trabalhos futuros podem explorar a integração de *caching* distribuído com `Redis` para consultas paginadas recorrentes.

Anexos

Todos os artefatos desenvolvidos, incluindo o código-fonte completo, modelos utilizados, configurações do banco de dados, arquivos de configuração do `RabbitMQ`, além de

exemplos de requisições e respostas simuladas podem ser acessados no repositório oficial do projeto:

<https://github.com/accj1990/Template.Storage.SQLServer>

O repositório contém:

- Projeto com scripts de inicialização de banco;
- Exemplos de uso implementados com exemplos de entrada/saída;
- Testes automatizados com cobertura por componente;
- Documentação técnica no formato `Markdown`.

Para executar o projeto localmente, basta seguir as instruções no arquivo `README.md`.

References

- [1] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002, define o Repository Pattern como um mediador entre as camadas de domínio e mapeamento de dados.
- [2] S. Overflow and D. Contributors. (2025) Dapper - a simple object mapper for .net. Referência oficial para o micro-ORM utilizado no projeto. [Online]. Available: <https://github.com/StackExchange/Dapper>
- [3] V. Vernon, *Implementing Domain-Driven Design*. Addison-Wesley, 2013, explora táticas de implementação de repositórios e o impacto na escalabilidade.
- [4] A. Silva and J. Santos, “Comparative analysis of orm and micro-orm performance in high-traffic web applications,” *Journal of Software Engineering and Applications*, vol. 16, pp. 45–60, 2023, estudo experimental que valida o uso de Dapper em sistemas com alta concorrência.